

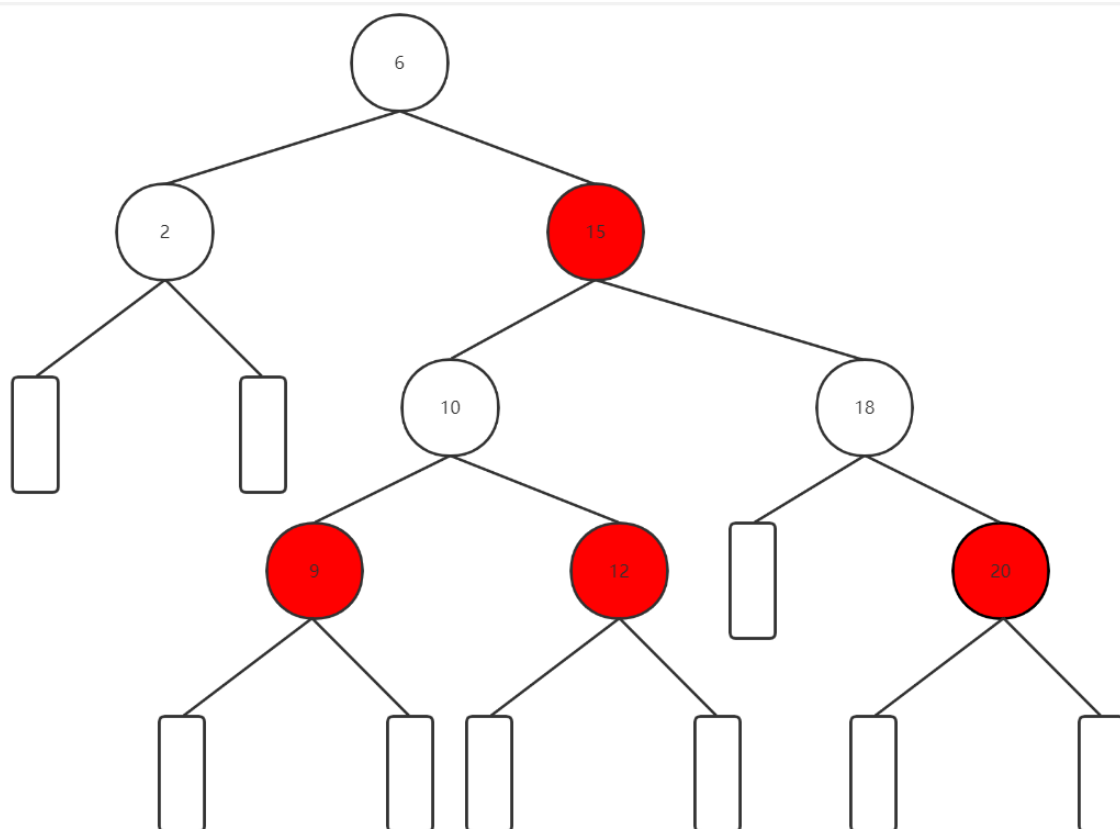
红黑树

现在，我们来看一看红黑树的精髓所在，我们来看一看，什么是红黑树的黑高，为什么要有红黑树，红黑树的旋转跟AVL有什么区别，如何去选择？红黑树是如何保持平衡的？直接进入正题

什么是红黑树

红黑树是一颗自平衡的二叉排序树，树上的每一个结点都遵循下面的规则（特别注意，这里的自平衡和平衡二叉树AVL的高度有区别）。我们再来看一看红黑树的定义：

- 1、每一个结点都有一个颜色，要么是红色，要么是黑色。
- 2、树的根结点为黑色
- 3、树中不存在两个相邻的红色节点（红色节点的父结点和孩子结点均不为黑色）
- 4、从任意一个结点出发，包括根结点，到其任何后代NULL结点（默认都是黑色啊）的每条路径都具有相同数量的黑色结点。



这就是一颗典型的红黑树，树中的每个结点的颜色要么是黑色，要么是红色；根结点 6 为黑色结点；树中不存在两个相邻的红色结点，比如结点 15 为红色结点，其父亲节点 6 与两个孩子结点就一定是黑色，而不能是红色；从结点到其后代的 NULL 结点的每条路径上具有相同数目的黑色结点，比如根结点 6 到其左子树的 NULL 结点包含三个黑色结点，到其右子树所有的 NULL 结点也包含三个黑色结点。可能还不够清晰，为此我对上图做了修改为所有默认为黑色的 NULL 结点给了一个标记。

现在解释规则的第四条简直不能再清晰了！比如根结点 6 到 NULL 结点 a 的路径 $6 \rightarrow 2 \rightarrow a$ 上的黑色结点为 3 个，从根结点 6 到结点 c 的路径 $6 \rightarrow 15 \rightarrow 10 \rightarrow 9 \rightarrow c$ 中包含的黑色结点个数也是 3 个，同理从根结点 6 到其他所有 NULL 结点的黑色结点数都是 3。再举个栗子，从红色结点 15 到 NULL 结点 d 的路径 $15 \rightarrow 18 \rightarrow g$ 包含 2 个黑色结点，到 NULL 结点 c 的路径 $15 \rightarrow 10 \rightarrow 9 \rightarrow c$ 也包含黑色结点 2 个，从结点 15 到其所有后代的 NULL 结点的黑色结点数目都是 2。

为什么要有红黑树

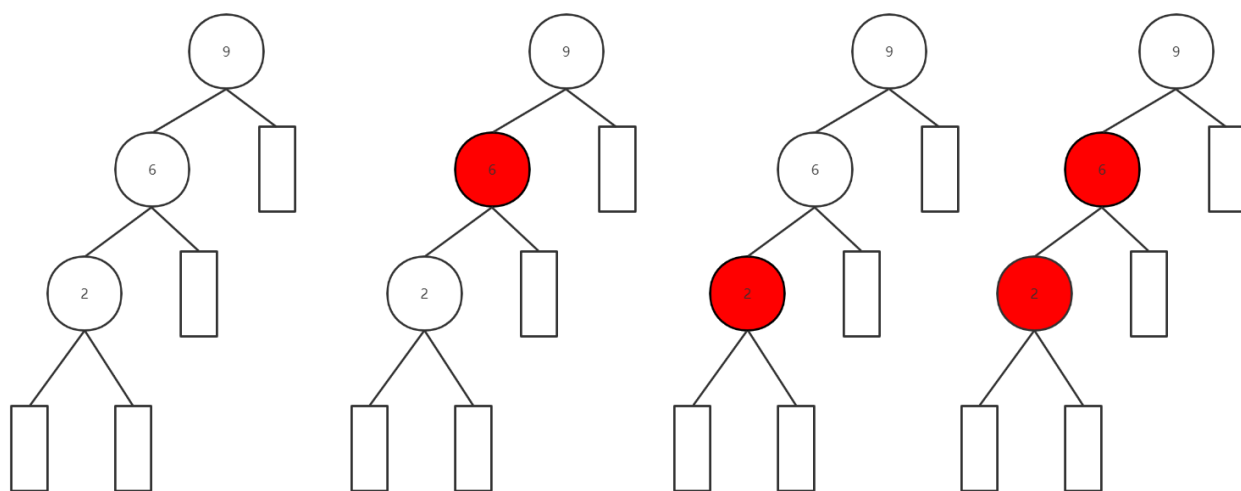
大多数二叉排序树 BST 的操作（查找、最大值、最小值、插入、删除等等）都是 $O(h)$ 的时间复杂度， h 为树的高度。但是对于斜树而言（BST 极端情况下出现），BST 的这些操作的时间复杂度将达到 $O(n)$ ， n 就是结点数。为了保证 BST 的所有操作的时间复杂度的上限为 $O(\log n)$ ，就要想办法把一颗 BST 树的高度一直维持在 $\log n$ ，而红黑树就做到了这一点，红黑树的高度始终都维持在 $\log n$ ， n 为树中的顶点数目。

这个时候就有一个疑问，不对啊。AVL 树不也始终是一个均值么？

红黑树 RBT 与平衡二叉树 AVL 的比较

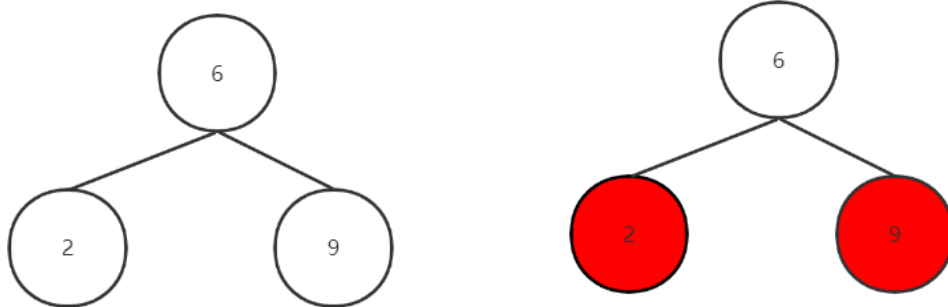
AVL 树比红黑树更加平衡，但 AVL 树在插入和删除的时候也会存在大量的旋转操作。所以当你的应用涉及到频繁的插入和删除操作，切记放弃 AVL 树，选择性能更好的红黑树；当然，如果你的应用中涉及的插入和删除操作并不频繁，而是查找操作相对更频繁，那么就优先选择 AVL 树进行实现。

一颗红黑树到底是如何保持平衡的呢？



举一个很简单但是很经典的例子，包含三个结点的单链是不可能出现在红黑树当中的。关于这一点，我们可以自己绘制一条单链，然后尝试为其着色，来判断。

从上图中可以发现，将根结点 9 涂黑色，其他结点分四种情况着色，结果都不满足红黑树的性质要求。唯一的办法就是调整树的高度



这就算我们对于红黑树的初探，然后我们来看两个重要的概念。

什么是一颗红黑树的黑高？

在一颗红黑树中，从某个结点 x 出发（不包含该结点）到达一个叶结点的任意一条简单路径上包含的黑色结点的数目称为 黑高，记为 $bh(x)$ 。所以我们发现，其实6和15的黑高是一样的，都是2。

计算结点 6 的黑高，从结点 6 到结点 c 的路径是 $6 \rightarrow 15 \rightarrow 10 \rightarrow 9 \rightarrow c$ ，其中黑色结点为 6、10、c，但是在计算黑高时，并不包含结点本身，所以从结点 6 到结点 c 的路径上的黑色结点个数为 2，那么 $bh(6)=2$ ；从结点 15 到结点 c 的路径为 $15 \rightarrow 10 \rightarrow 9 \rightarrow c$ ，其中黑色结点为 10、c，所以从结点 15 到结点 c 的路径上黑色结点数目为 2， $bh(15)=2$ 。

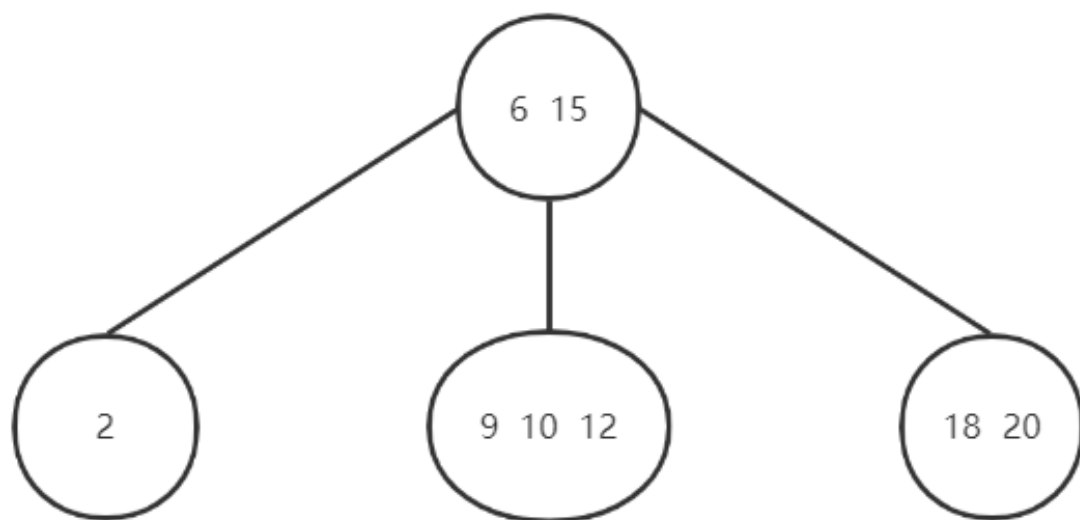
因为红黑树的黑高为其根结点的黑高。所以根据红黑树的性质3和性质4，一颗红黑树的黑高 bh 一定 $\geq h/2$ 。

Number of nodes from a node to its farthest descendant leaf is no more than twice as the number of nodes to the nearest descendant leaf.

从一个结点到其最远的后代叶结点的顶点数目不会超过从该结点到其最近的叶结点的结点数目的两倍。

其中黑高 bh 就表示从根结点 6 到离它最近的叶结点 2 包含的结点数 2，而 h 则表示从根结点 6 到其最远的叶结点 9 所包含的结点数目 4，显然这一公式是合理的。

引出来一个道理：一颗有 n 个结点的红黑树的高度 $h \leq 2\lg(n+1)$ 。



这就合并成了一颗2-3-4树，这棵树中的每一个结点有2、3、4个孩子结点，而一颗2-3-4树的叶结点有着相同的深度 h' 。

也正是基于这个，所以对于有 n 个结点的红黑树而言，不论查找、删除、最大值、最小值等的时间复杂都平均下来了，也就是 $O(\log n)$ 。

红黑树的插入

其实红黑树的操作也很简单，就是比AVL多了一个着色的操作。

在AVL中，我们通过左旋和右旋来调整由于插入和删除所造成的不平衡的问题。在红黑树中，我们使用两种方式：

- 1、重新着色
- 2、旋转

当红黑树中出现不平衡的状态，我们首先会考虑重新着色，如果重新着色依旧不能使红黑树平衡，那么就考虑旋转。插入操作主要有两种情况，具体取决于叔叔结点的颜色。如果叔叔结点是红色的，我们会重新着色。如果叔叔结点是黑色的，我们会旋转或者重新着色，或者两者都考虑。

假设 x 是新插入的一个结点。

- 1、进行标准的BST插入并将新插入的结点设置为红色
- 2、如果 x 是根结点，将 x 的颜色转化为黑色（整棵树的黑高增加1）
- 3、如果 x 的父结点 p 不是黑色并且 x 不是根结点，则：
 - 1)、如果 x 的叔叔结点 u 是红色；
 - 2)、如果 x 的叔叔结点 u 是黑色，则对于 x 、 x 的父结点 p 和 x 的爷爷结点 g 有四种情况：

LL (p是g的左孩子且x是p的左孩子)

LR (p是g的右孩子且x是p的右孩子)

RR (p是g的右孩子且x是p的右孩子)

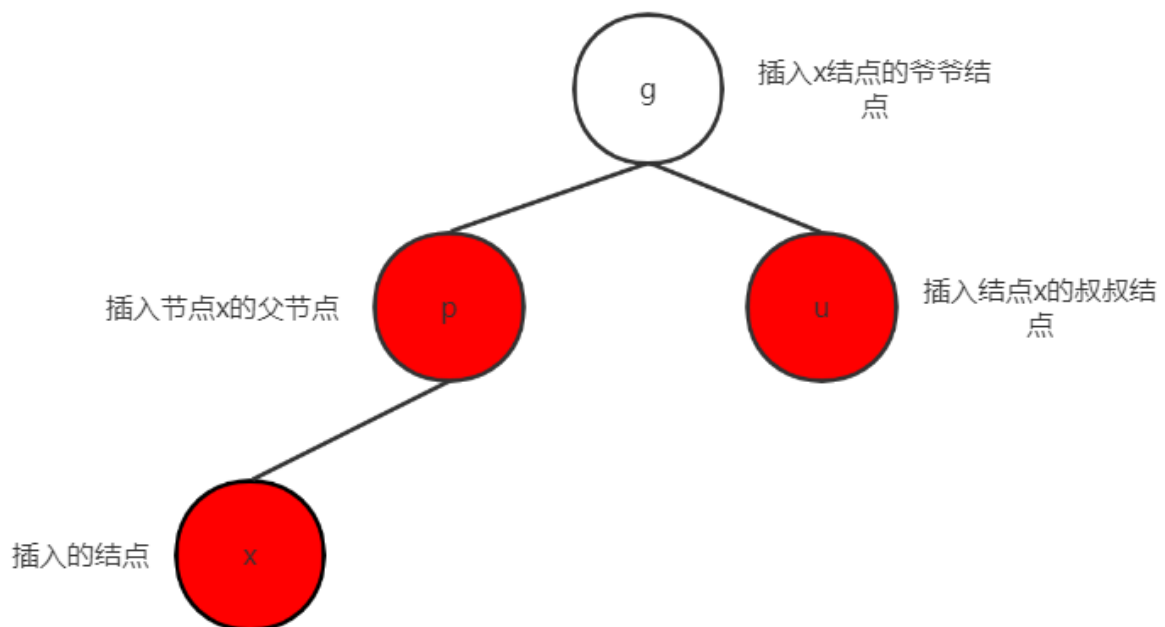
RL (p是g的右孩子且x是p的左孩子)

将插入结点x的父结点p和叔叔结点u的颜色变成黑色

将x的爷爷结点g设置为红色

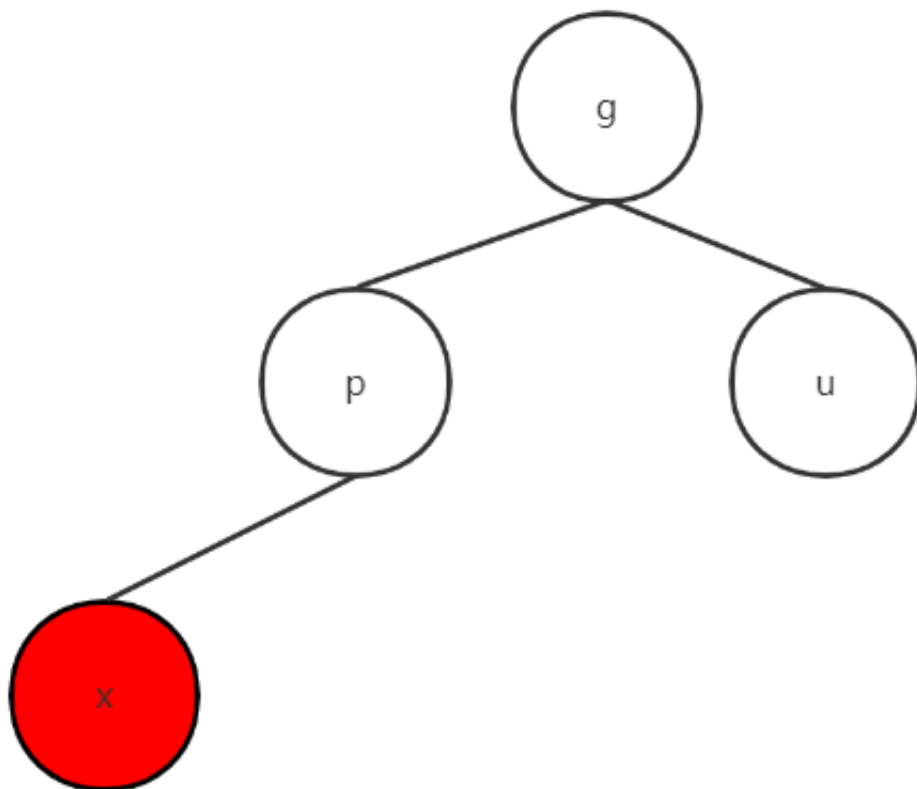
将g看作是x，对于新的x重复2、3两步

插入结点x的叔叔结点u是红色

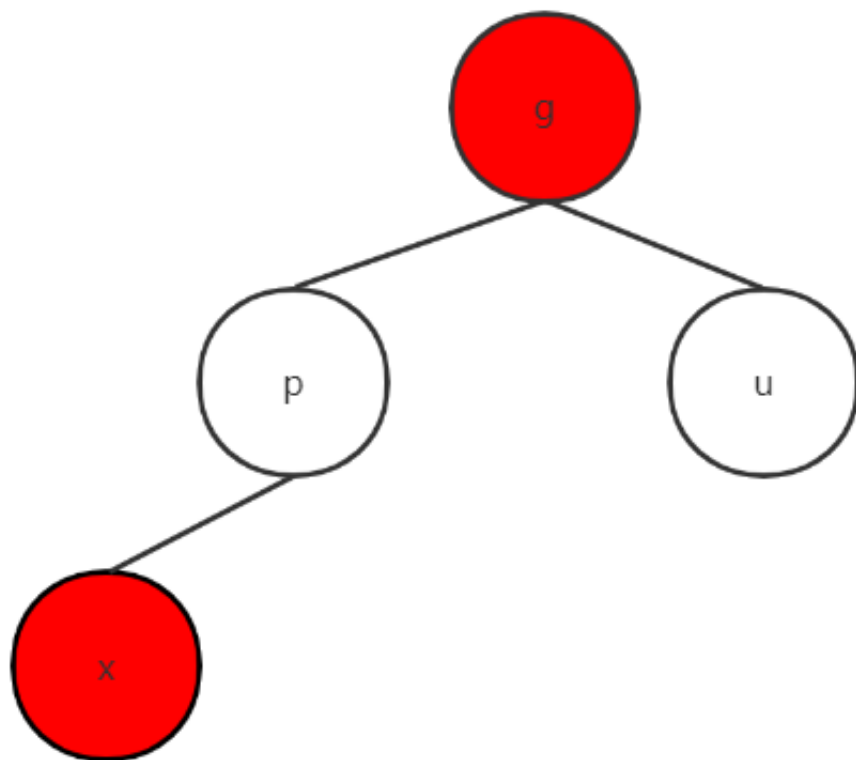


对于新插入结点 x，我们执行标准的 BST 插入之后，将插入结点 x 着色为红色；如果插入结点不是根结点，且x的父结点 p 为红色结点，分为 1) 和 2) 两种情况处理，我们先看的是 1) 的情况：x 的叔叔结点 u 为红色，如下图所示：

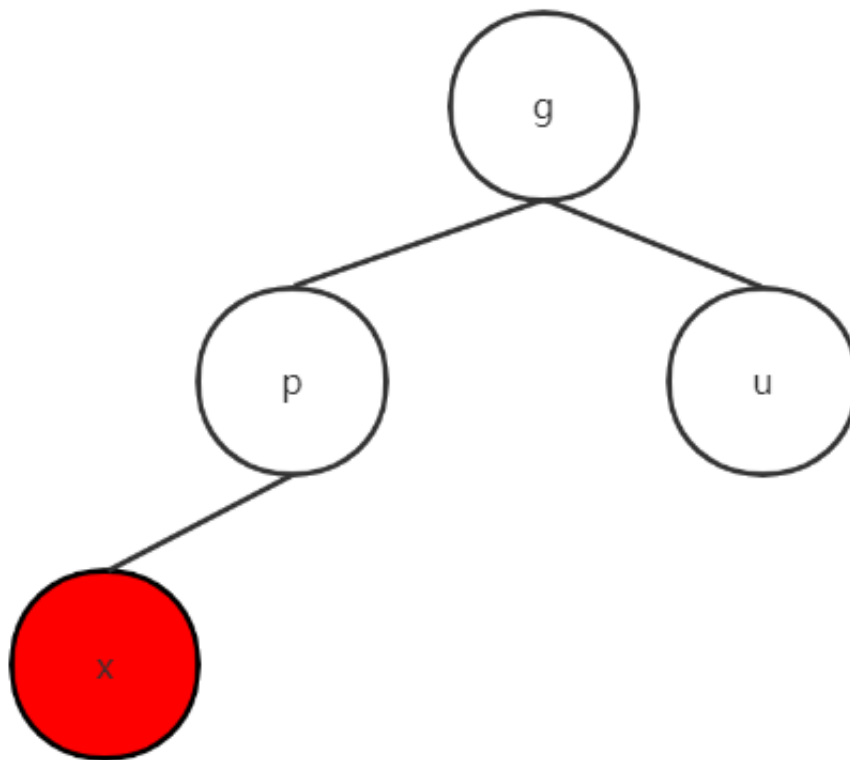
第一步：将父亲结点p和叔叔结点u都设置为黑色



第二步：将g的颜色设置为红色



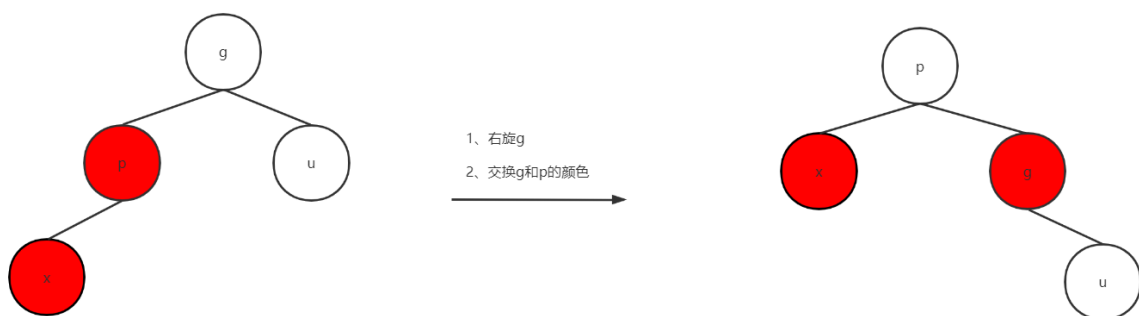
第三步：针对于g结点，在执行第二，第三步。



插入结点x的叔叔结点u为黑色

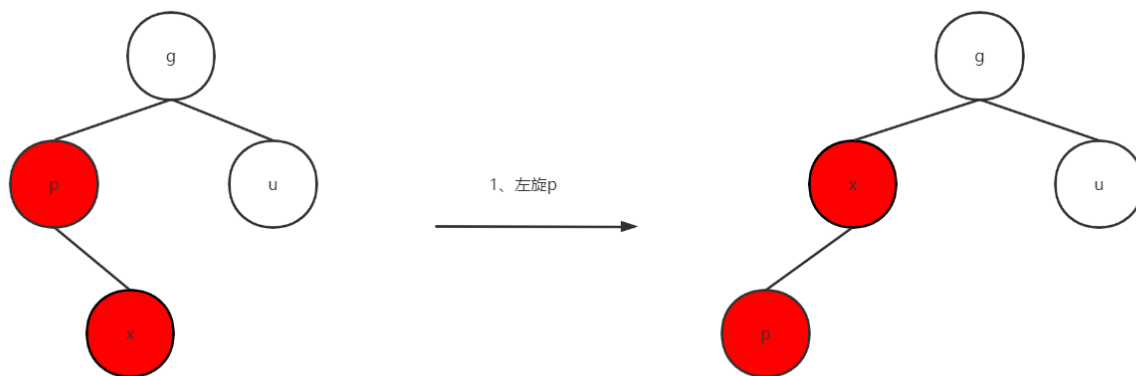
当插入结点x的叔叔结点为黑色的时候，根据插入接待你x、x的父结点p和x的爷爷结点g可能出现的位置关系，分为四种情况。

LL

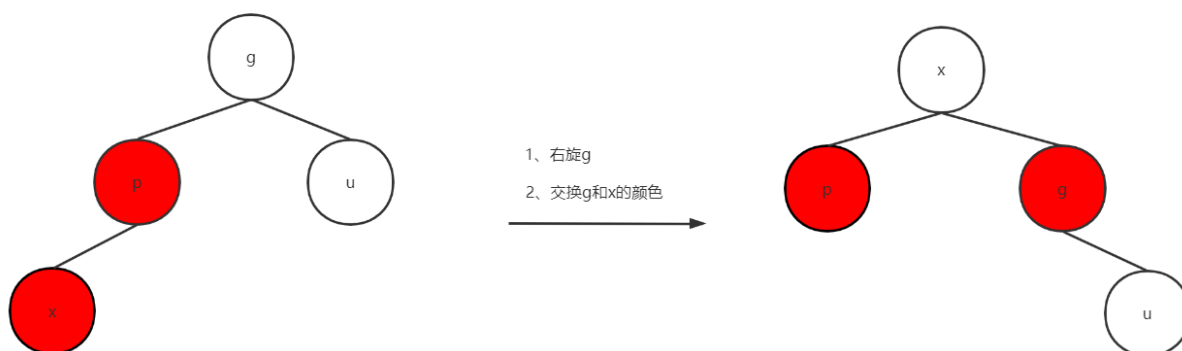


LR

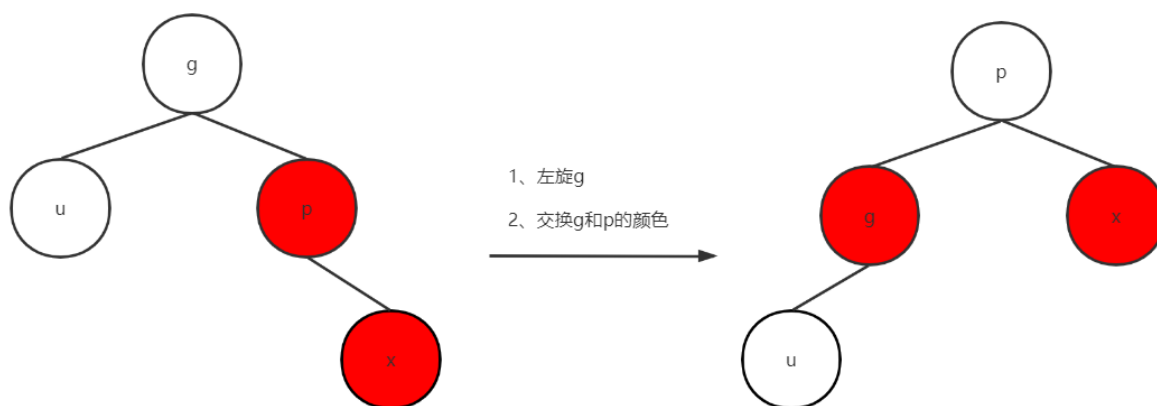
首先通过左旋p转化为LL的情况：



然后按照LL的情况处理：

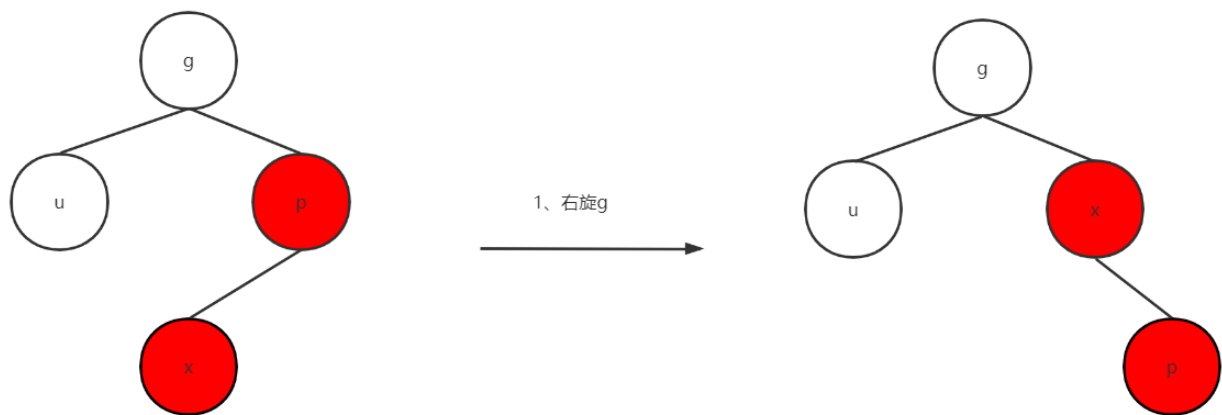


RR

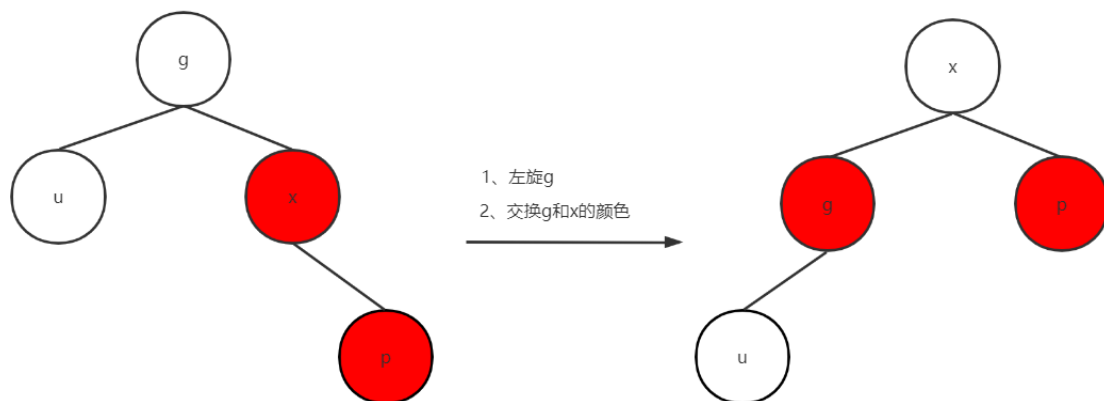


RL

先右旋转化成RR的情况



然后按照RR的情况处理



红黑树的删除

说起红黑树的删除操作，就不得不提我们讲的红黑树的插入。与红黑树的插入操作类似，红黑树的删除也是重新着色和旋转来保证每一次删除操作后依旧满足红黑树的属性。

在插入操作中，通过判断插入结点 x 的叔叔结点 u 的颜色来确定恰当的平衡操作。而删除操作中，是通过检查兄弟结点的颜色来决定恰当的平衡操作。

红黑树中插入一个结点最容易出现两个连续的红色结点，违背红黑树的性质3（红黑树中不存在两个相邻的红色结点）。而删除操作，最容易造成子树黑高（Black Height）的变化（删除黑色结点可能导致根结点到叶结点黑色结点的数目减少，即黑高降低）。

与插入操作相比，红黑树的删除操作相对复杂一点，但多点儿耐心，还是没有问题的。为了解除删除操作，我们先来看一个双黑（Double Black）的概念。

当删除结点 v 是黑色结点，且其被其黑色子节点替换时，其子结点就被标记为双黑。

所以说，删除操作最主要的任务就是将转化为双黑结点转换为我们普通的黑色结点。

删除操作总纲

删除操作总体上分为三步，我们先提高挈领地看一下，有个宏观概念，然后步步为营，攻陷删除。

首先我们假定要删除的结点为 v ， u 是用来替换 v 的孩子结点（注意，当 v 是叶结点时， u 是 NULL 结点，且 NULL 结点我们还是当做黑色结点处理）。

删除操作总纲：

- 1、执行标准的BST的删除操作
- 2、简单情况：u或者v是红色
 - 1) u是双黑结点
 - 2) 当前结点u是双黑结点且不是根结点
 - a) u的兄弟结点s是黑色且s的孩子结点至少有一个是红色（LL、LR、RR、RL）
 - b) u的兄弟结点s是黑色且它的两个孩子都是黑色
 - c) u的兄弟结点s是红色（s是其父结点p的左孩子、s是其父结点的右孩子）
 - 3) 当前结点u是双黑结点且是根结点

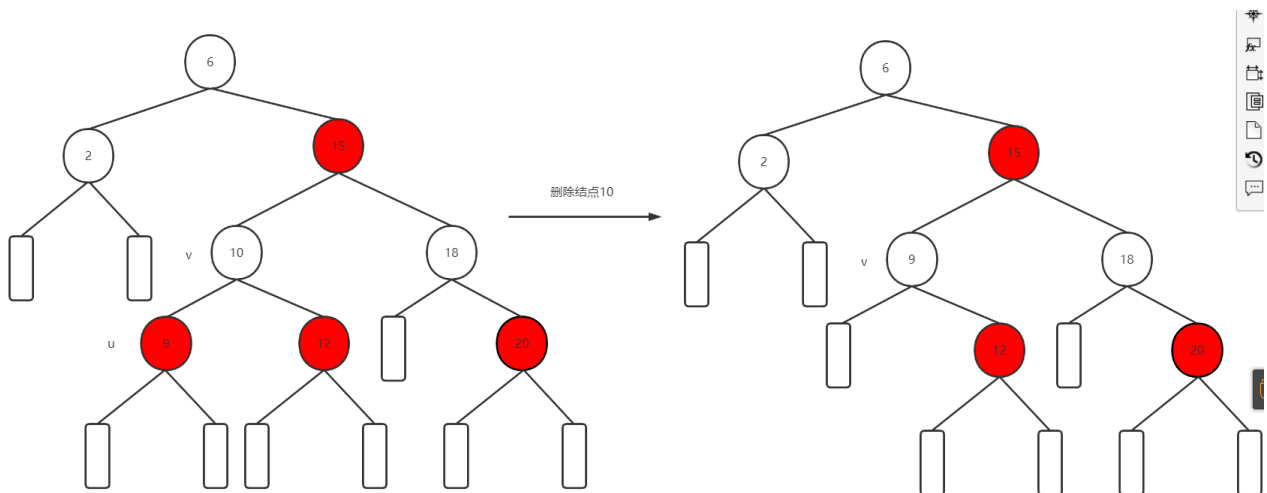
1、执行标准的BST删除操作

在标准的 BST 删除操作中，我们最终都会以删除一个叶子结点或者只有一个孩子的结点而结束（对于内部节点，就是要删除结点左右孩子都存在的情况，最终都会退化到删除结点是叶子结点或者是只有一个孩子的情况）。所以我们仅需要处理被删除结点是叶结点或者仅有一个孩子的情况。

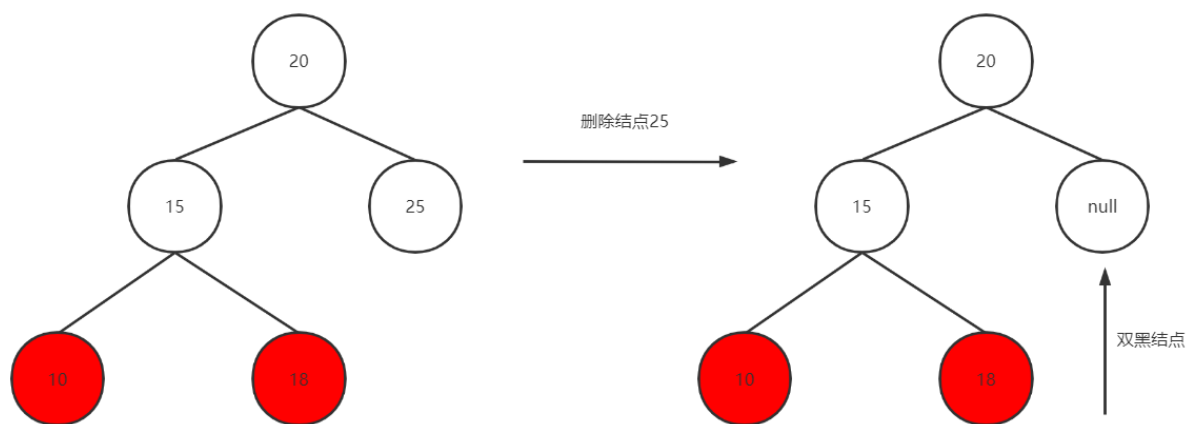
2、简单情况：u或者v是红色

如果 u 或者 v 是红色，我们将替换结点 v 的结点 u 标记为黑色结点（这样黑高就不会变化）。注意这里是 u 或者 v 是红色结点，因为在一棵红黑树中，是不允许有两个相邻的红色结点的，而结点 v 是结点 u 的父结点，因此只能是 u 或者 v 是红色结点。

删除结点 v 为黑色结点 10，替换结点 v 的结点 u 为红色结点 9 的情况：

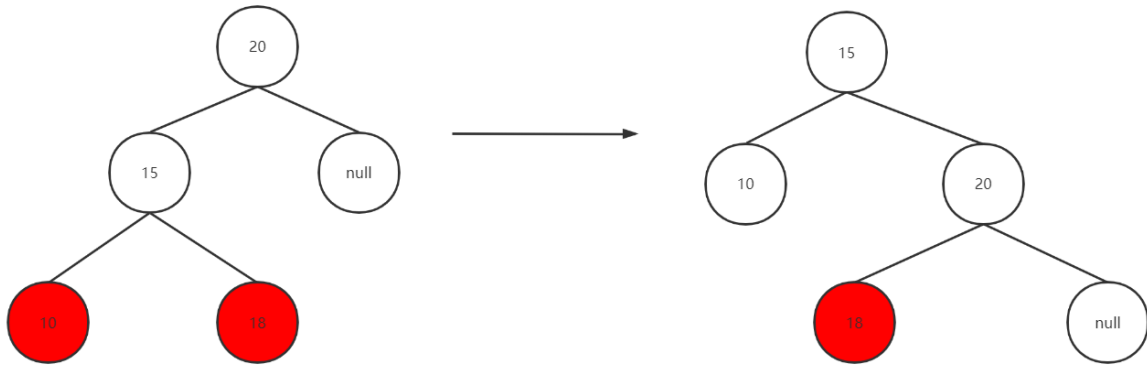


删除结点v为红色结点20，替换结点v的结点u为黑色NULL结点的情况：



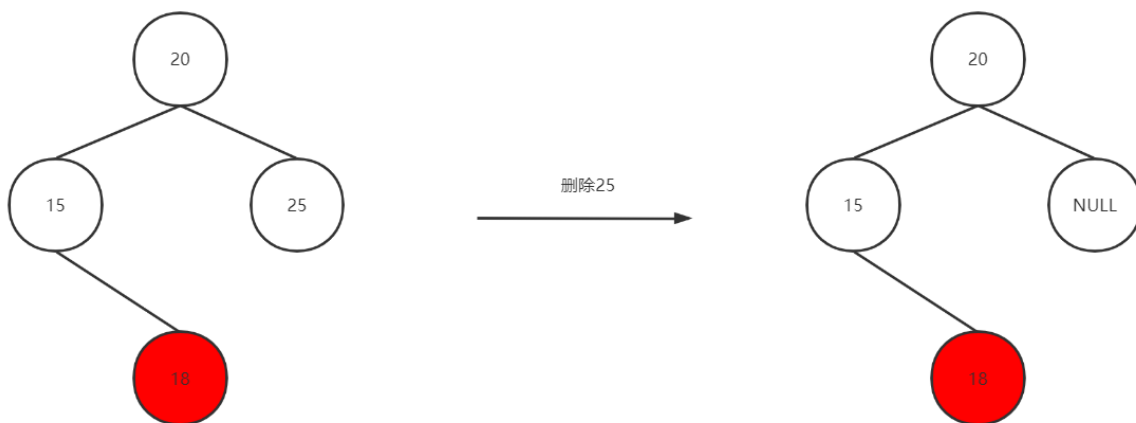
删除结点 25，用结点 25 的 NULL 结点 替换结点 25，产生一个双黑结点 u，双黑结点 u 的兄弟结点 s 为 15，结点 s 是其父结点 20 (p) 的左孩子，其左孩子 10 (r) 正好是红色结点。即为 LL 情况。

s 的左孩子 r 颜色设置为 s 的颜色，s 的颜色设置为父结点 p 的颜色，然后右旋 p 结点。



LR (s是p的左孩子，r是s的右孩子，或者s的两个孩子都是红色)

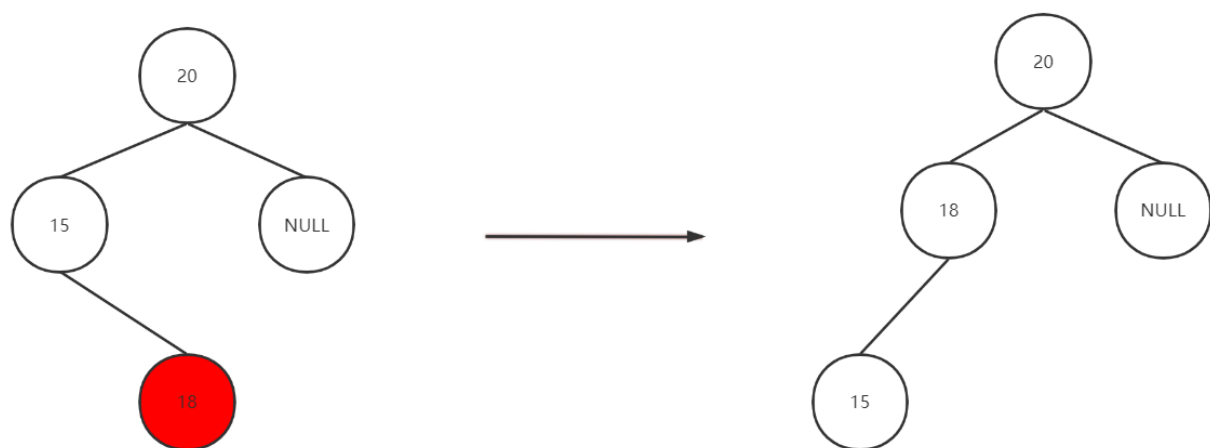
删除结点25，不过结点25的兄弟结点15只有一个右孩子18



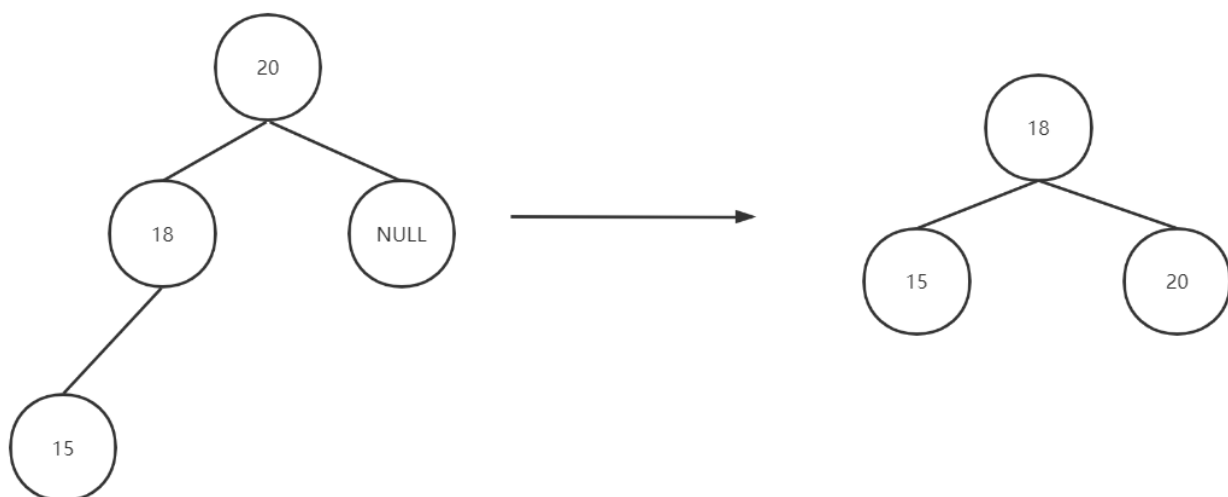
将结点r的颜色设置为p的颜色。



左旋结点15 (s)

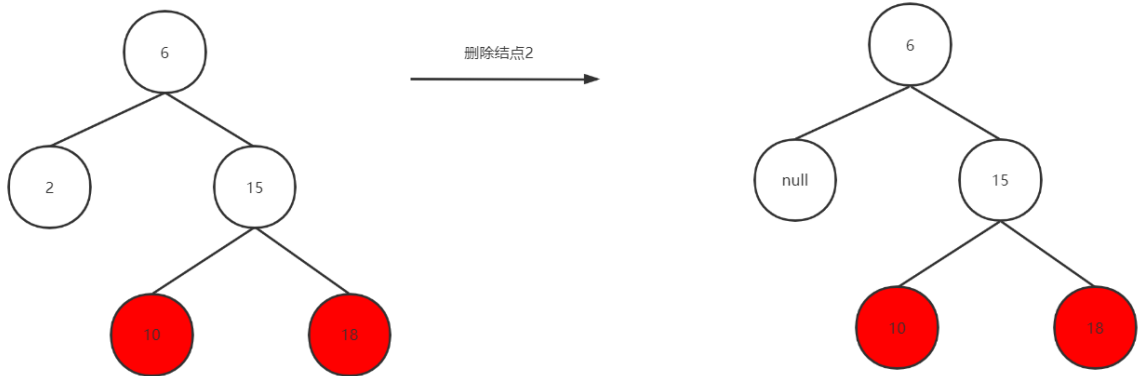


右旋结点20 (p) , p的颜色设置为黑色, 双黑变单黑

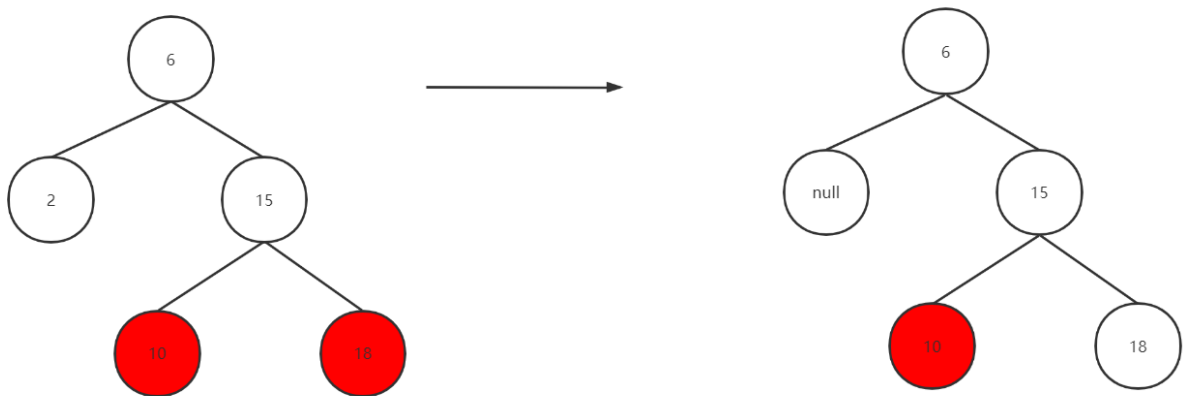


RR (s 是 p 的右孩子, r 是 s 的右孩子, 或者 s 的两个孩子都是红色结点) :

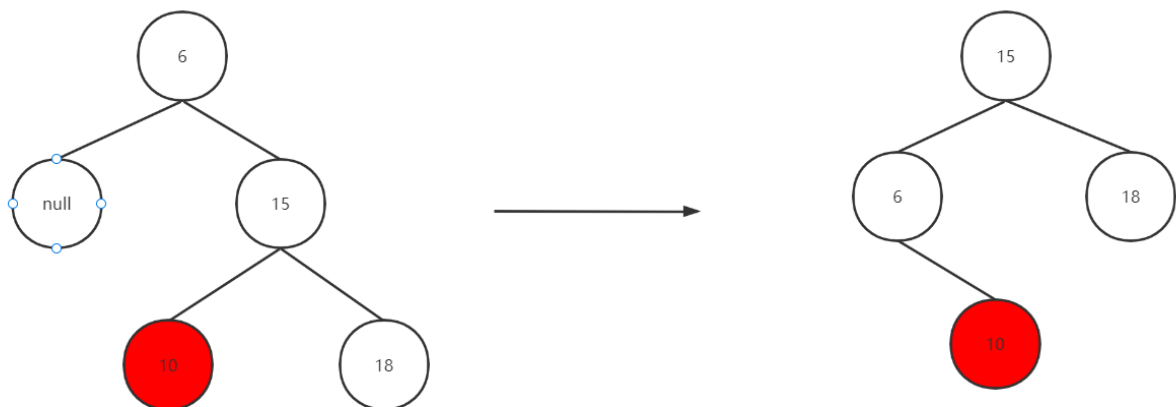
删除结点 2, 用结点 2 的 NULL 结点 a 替换结点 2, 产生一个双黑结点 u, 双黑结点 u 的兄弟结点 s 为 15, 结点 s 是其父结点 6 (p) 的右孩子, 其右孩子 18 (r) 正好是红色结点。即为 RR 情况 (仔细观察其实和 LL 情况是对称的)。



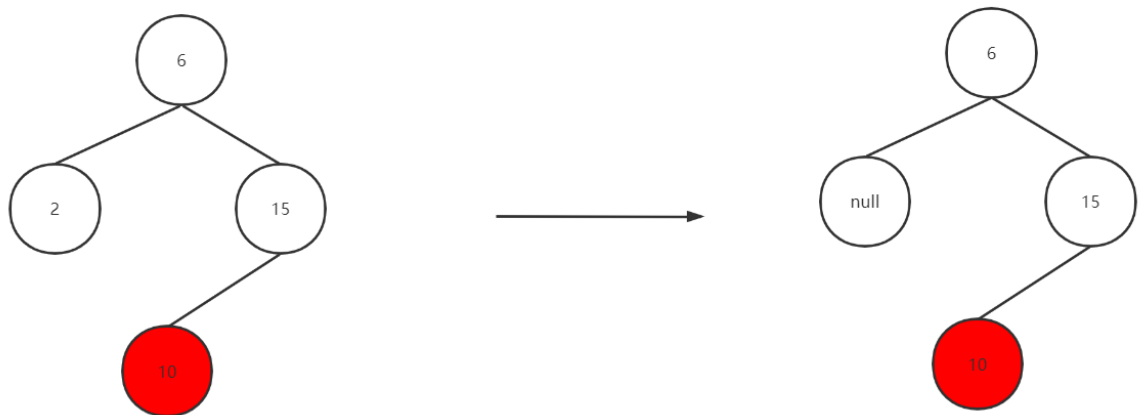
r的颜色变为s的颜色, s的颜色变为p的颜色。



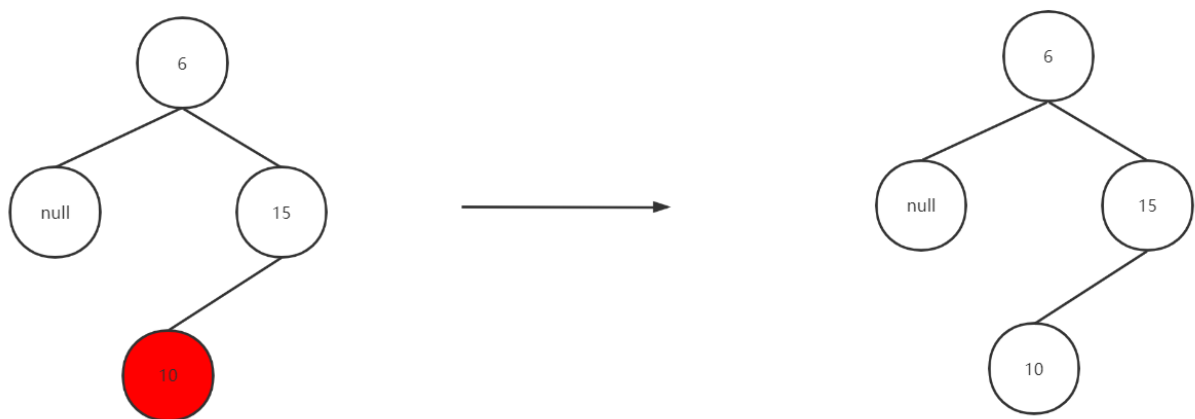
左旋p, p的颜色设置为黑色, 双黑变单黑



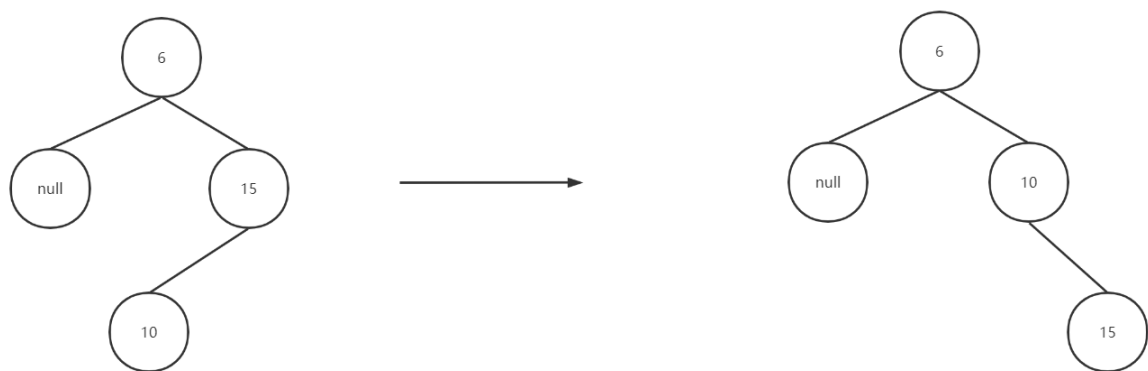
RL情况 (s 是 p 的右孩子, r 是 s 的左孩子, 或者 s 的两个孩子都是红色结点) : 该情况与 LR情况是对称的



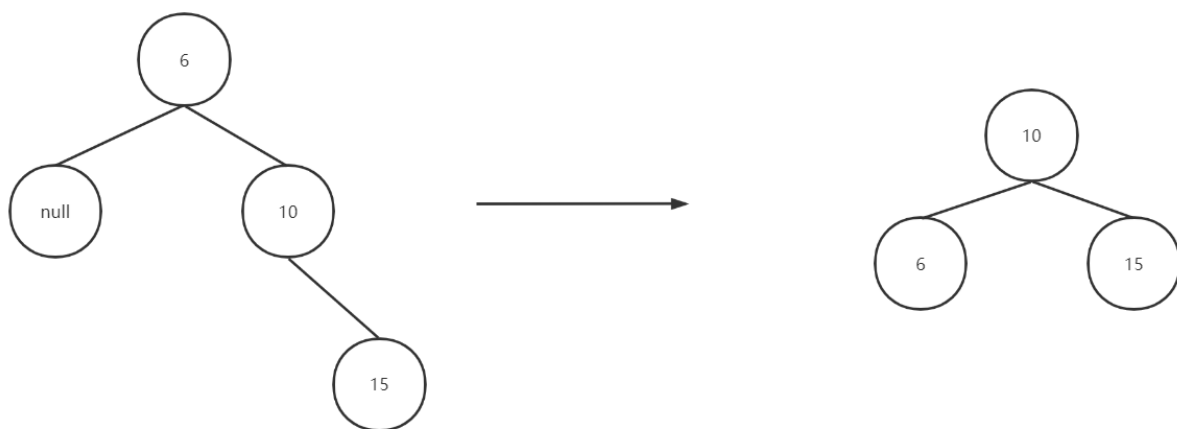
结点r的颜色变为p的颜色



右旋结点15



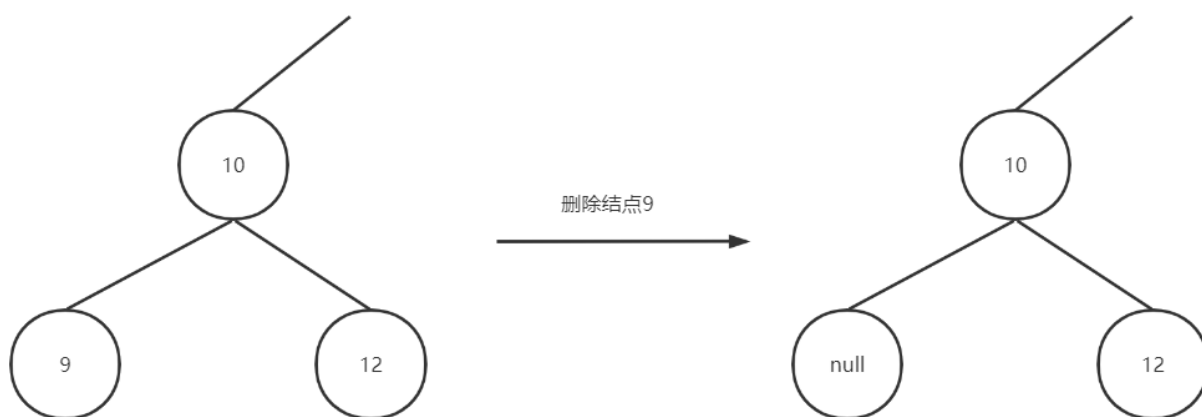
左旋结点6 (p) , p的颜色设置为黑色, 双黑变单黑



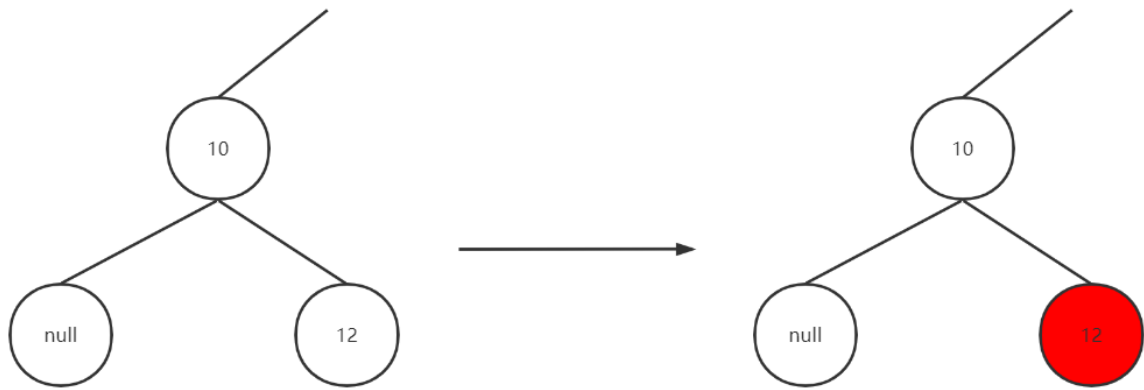
u 的兄弟结点 s 是黑色且 s 的两个孩子结点都是黑色

对于这种情况需要递归地进行处理，如果删除结点后得到的双黑结点的父结点此时为黑色，则结点 u 变单黑，且结点 u 的父结点 p 变双黑，然后对结点 u 的父结点 p 继续进行处理，直到当前处理的双黑结点的父结点为红色结点，此时将双黑结点的父结点设置为黑色，双黑结点变为单黑结点（红色 + 双黑 = 单黑）。

假设以 10 为根结点的子树为整棵树的左子树，删除结点 9，产生双黑结点 c 且其兄弟结点 12 (s) 为黑色，兄弟结点的左右孩子均为黑色。

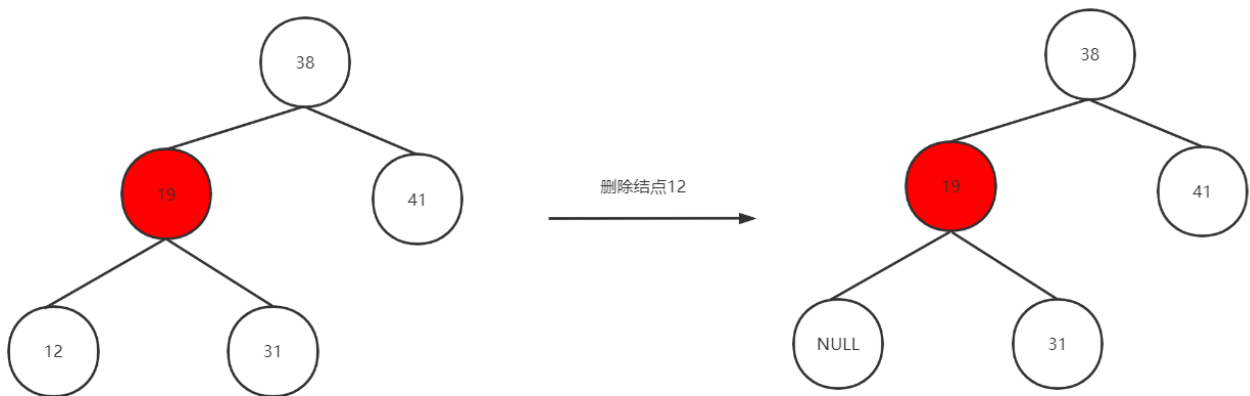


此时双黑结点的兄弟结点 12 变为红色结点，然后将 u 的父结点 10 变为双黑结点，一直向上判断。

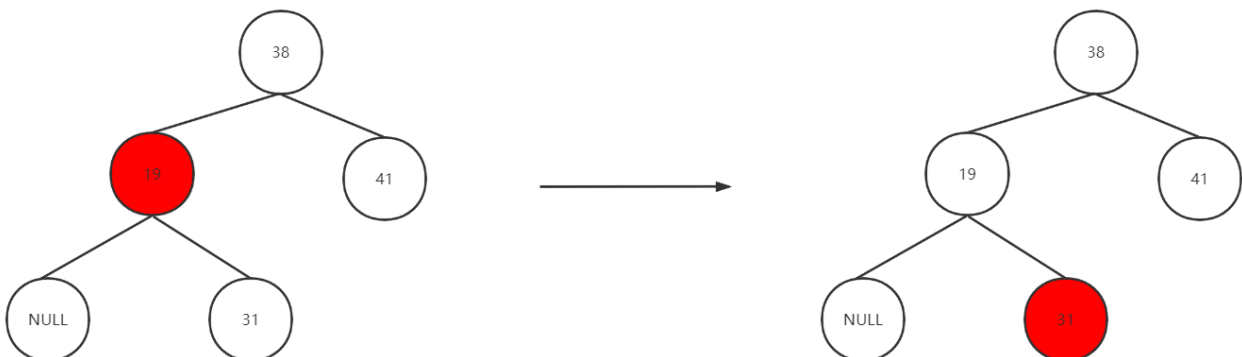


那么这个过程什么时候结束呢？

如下图，删除结点12，得到一个双黑结点u，双黑结点的兄弟结点31及兄弟结点的孩子结点均为黑色，且双黑结点的父结点19为红色结点，刚好是不再继续向上判断的情况：



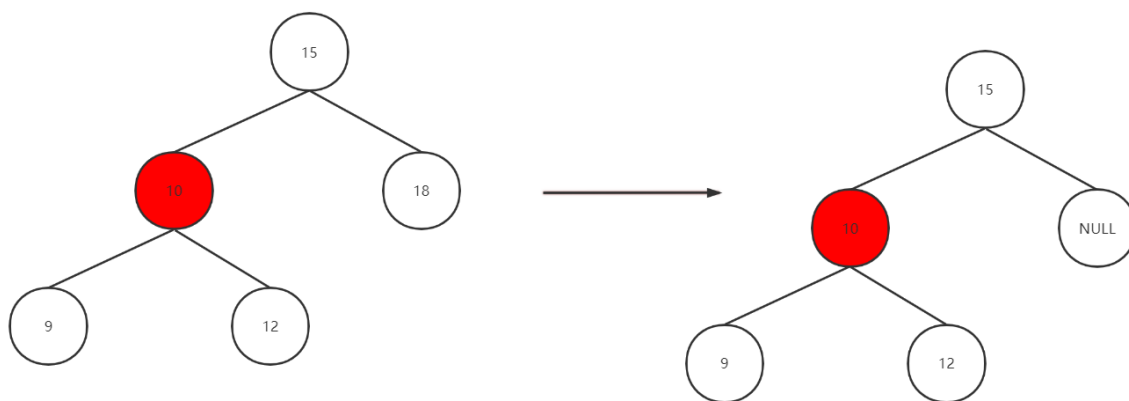
此时只需要将结点 u 的兄弟结点 31 的颜色变为红色，双黑结点 u 的父结点 19 由红色变为黑色结点，双黑结点 u 变为单黑结点。



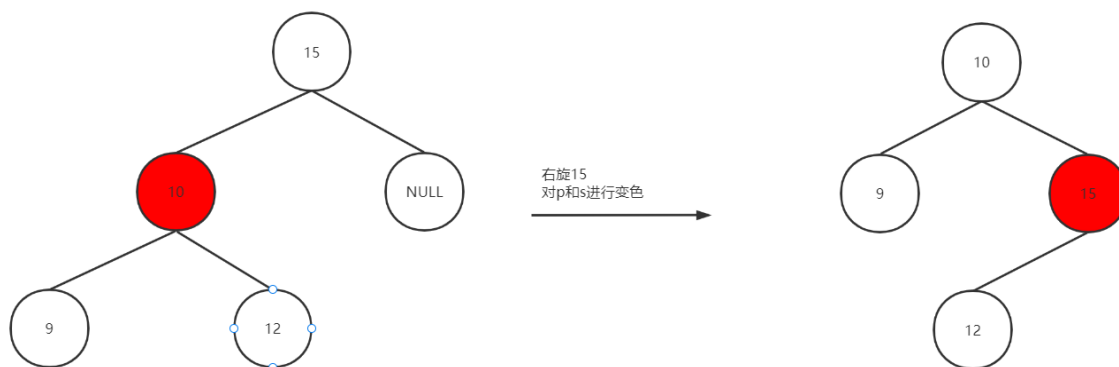
u的兄弟结点s是红色结点

当前 u 的兄弟结点 s 是红色结点时，通过旋转操作将 u 当前的兄弟结点向上移动，并对 u 的父结点和其旋转前的兄弟结点重新着色，接着继续对结点 u 旋转后的兄弟结点 s 进行判断，确定相应的平衡操作。旋转操作将 u 的兄弟结点情况又会转换为前面刚提到的3.2 (a) 和 (b) 的情况。根据兄弟结点 s 是父结点 p 的左右孩子又分为两种情况。

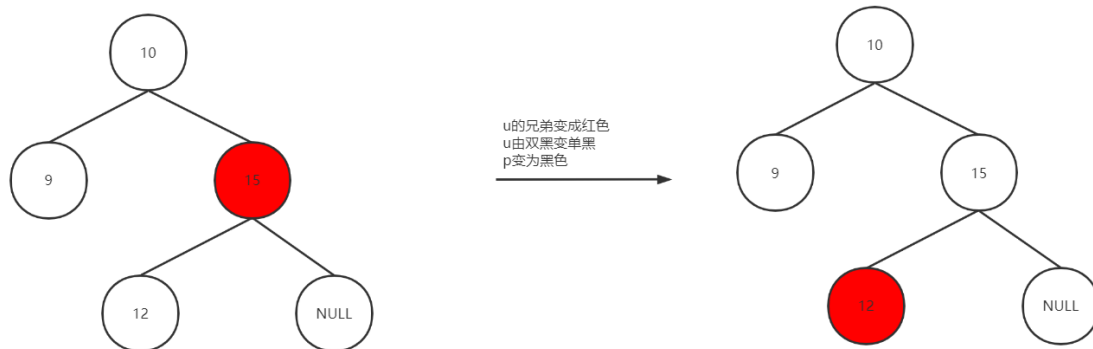
情况一：u 的兄弟结点 s 是父结点 p 的左孩子 ,对结点 p 进行右旋操作。



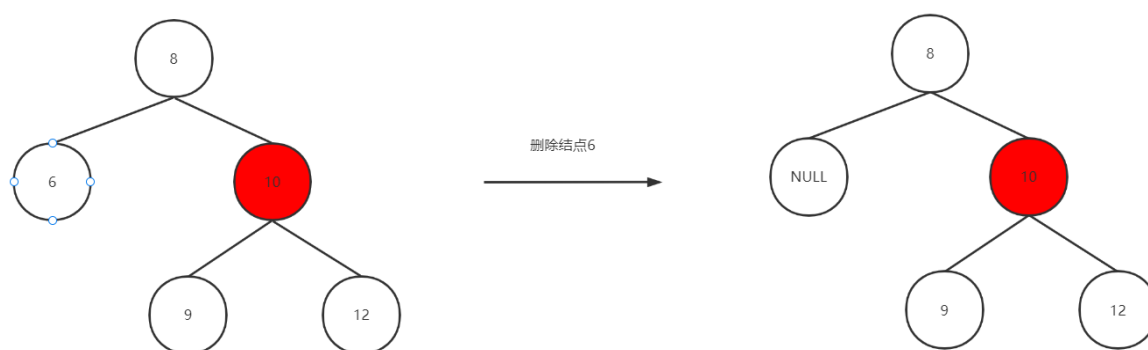
删除结点 18，产生一个双黑结点 u，且 u 的兄弟结点 s 是红色，兄弟结点 s 是其父结点的左孩子，接着就是对其父结点 15 进行右旋操作。



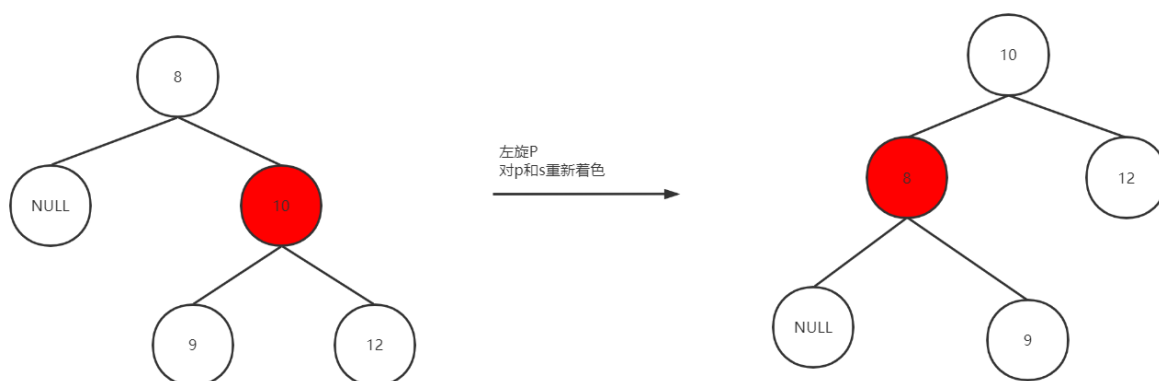
对结点 15 进行右旋操作，并且对旋转前的 p 和 s 进行重新着色后，继续对双黑结点旋转后的兄弟结点进行判断，发现此时正好和 3.2 (b) 的情况是一样，进行相应处理，如下图所示。



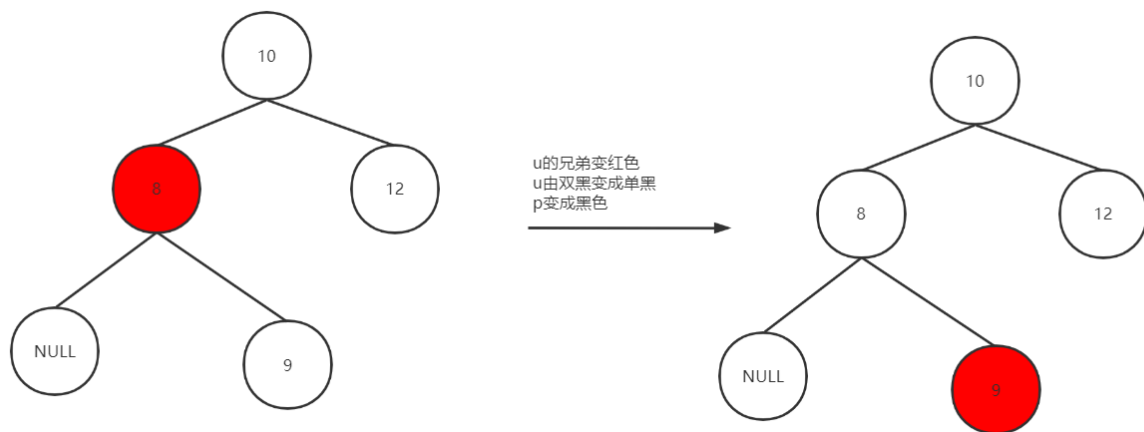
情况二：u 的兄弟结点 s 是父结点 p 的左孩子 ,对结点 p 进行左旋操作（这种情况与上面的是对称的）。



删除结点 6，产生一个双黑结点 u，且 u 的兄弟结点 10 (s) 为红色，s 是父结点 p 的右孩子，左旋P



对双黑结点 u 旋转后的兄结点继续判断：



3.3 当前结点u是双黑结点且是根结点

当前结点 u 是双黑结点且是根结点时，直接将双黑结点变为单黑结点，整颗红黑树的黑高减1.

红黑树与AVL树的比较

红黑树

红黑树中的每个结点需要一个存储位表示结点的颜色，可以是红或黑。通过对任何一条从根到叶子的路径上各个结点着色方式的限制，红黑树确保对于每一个结点其到叶子结点的最长路径不会超过最短路径的两倍，因此，红黑树是一种弱平衡二叉树（由于是弱平衡，可以看到，在相同结点的情况下，AVL树的高度 $\leq$ 红黑树），相对于要求严格的AVL树来说，它的旋转次数少，所以对于插入，删除操作较多的情况下，使用红黑树。